https://app.notion.com/p/Kanban-Menu-Maker-by-Qwenta-38e25e1b8362801a8f3dd734e98a37bd

# Kanban - Menu Maker by Qwenta

**Tâches techniques Menu Maker**

| Tâches | État | User Story liée | Priorité | Assigné à | Story points | Critères de succès | Spécifications techniques |
|---|---|---|---|---|---|---|---|
| Intégrer la landing page non connectée | À faire | Landing non connectée | P1 | Développeur front-end | 3 | La landing page est intégrée conformément à la maquette desktop. Les sections principales sont visibles, la navigation fonctionne et les boutons d'appel à l'action permettent d'accéder au parcours prévu. | React, composants réutilisables, CSS conforme à la charte graphique, React Router pour les liens vers les pages publiques et la connexion. |
| Créer les pages publiques Tarifs et Informations légales | À faire | Tarifs / Informations légales | P2 | Développeur front-end | 2 | Les pages Tarifs et Informations légales sont accessibles depuis la navigation publique. Leur contenu est lisible, structuré et conforme à la maquette desktop. | React, React Router, composants de page statiques, HTML sémantique, CSS conforme à la charte graphique. |
| Créer la page de connexion | À faire | Page login | P1 | Développeur front-end | 3 | La page de connexion est accessible depuis la navigation publique. Le formulaire est lisible, utilisable au clavier et prévoit l'affichage des erreurs de saisie. | React, React Router, formulaire contrôlé, HTML sémantique, accessibilité clavier, préparation de l'intégration Auth0. |
| Intégrer l'authentification avec Auth0 | À faire | Page login | P1 | Développeur back-end | 5 | L'utilisateur peut se connecter via Auth0. Après connexion, il accède à l'espace privé de l'application. Les informations d'identification sont gérées par Auth0 et ne sont pas stockées directement dans l'application. | Auth0, configuration d'une application web, gestion des tokens, connexion entre React et l'API Express.js, vérification de l'identité côté back-end. |
| Protéger les routes privées de l'application | À faire | Page login / Dashboard | P1 | Développeur front-end | 3 | Les pages privées de l'application ne sont accessibles qu'aux utilisateurs connectés. Un utilisateur non connecté est redirigé vers la page de connexion. | React Router, routes protégées, vérification de l'état de connexion Auth0, redirection vers la page login si l'utilisateur n'est pas authentifié. |
| Créer la structure du dashboard | À faire | Dashboard | P2 | Développeur front-end | 3 | Le dashboard est accessible après connexion. Il présente les principales actions disponibles : créer un menu, consulter les menus précédents et accéder aux informations utilisateur. | React, React Router, layout privé, composants de navigation interne, affichage conditionné à l'utilisateur connecté. |
| Créer le modèle de données Menu | À faire | Création de menu | P1 | Développeur back-end | 3 | La structure de données permet de créer, sauvegarder et retrouver un menu associé à un restaurateur. | PostgreSQL, modèle relationnel, table menus, relation avec l'utilisateur ou le restaurateur, dates de création et modification. |
| Créer le modèle de données Catégorie | À faire | Catégorie de plat | P1 | Développeur back-end | 2 | Les catégories de plats peuvent être associées à un menu et ordonnées selon les besoins du restaurateur. | PostgreSQL, table categories, relation avec la table menus, champ d'ordre d'affichage. |
| Développer l'API de gestion des catégories | À faire | Catégorie de plat | P1 | Développeur back-end | 5 | L'API permet de créer, modifier, supprimer et récupérer les catégories d'un menu. | Express.js, API REST, endpoints CRUD, PostgreSQL, vérification des droits utilisateur. |
| Créer l'interface de gestion des catégories | À faire | Catégorie de plat | P1 | Développeur front-end | 3 | L'utilisateur peut ajouter, modifier et supprimer des catégories depuis l'interface de création de menu. | React, formulaire contrôlé, appels API, gestion d'état, affichage dynamique des catégories. |
| Créer le modèle de données Plat | À faire | Création de plat | P1 | Développeur back-end | 3 | La structure de données permet d'enregistrer les plats avec leur nom, leur description, leur prix et leur catégorie. | PostgreSQL, table dishes, relation avec categories, champs nom, description, prix, ordre d'affichage. |
| Développer l'API de gestion des plats | À faire | Création de plat | P1 | Développeur back-end | 5 | L'API permet de créer, modifier, supprimer et récupérer les plats associés aux catégories d'un menu. | Express.js, API REST, endpoints CRUD, PostgreSQL, validation des données côté serveur. |
| Créer le formulaire d'ajout et modification de plat | À faire | Création de plat | P1 | Développeur front-end | 5 | L'utilisateur peut saisir ou modifier un plat avec un nom, une description, un prix et une catégorie. Les erreurs de saisie sont visibles. | React, formulaire contrôlé, validation des champs, appels API, messages d'erreur accessibles. |
| Créer l'interface de prévisualisation du menu | À faire | Création de plat / Style de menu | P1 | Développeur front-end | 5 | L'utilisateur peut visualiser en temps réel le rendu du menu avec ses catégories, ses plats et les informations saisies. | React, composants de prévisualisation, gestion d'état locale, affichage dynamique des catégories et plats. |
| Développer la personnalisation du style du menu | À faire | Personnaliser le style de menu | P1 | Développeur front-end | 5 | L'utilisateur peut personnaliser l'apparence du menu, notamment les couleurs, la typographie et le logo. | React, gestion d'état, upload de logo, styles dynamiques CSS, prévisualisation en temps réel. |
| Sauvegarder les préférences de style | À faire | Sauvegarder le style de menu | P1 | Développeur back-end | 3 | Les préférences de style du menu sont sauvegardées et retrouvées lors de la consultation ou modification du menu. | Express.js, PostgreSQL, API REST, stockage des préférences de couleur, police et logo. |
| Générer l'export PDF du menu | À faire | Exporter le menu en PDF | P1 | Développeur back-end | 8 | L'utilisateur peut générer et télécharger un PDF fidèle au rendu du menu créé dans l'application. | Express.js, Puppeteer, génération PDF côté serveur, template HTML/CSS, endpoint dédié à l'export PDF. |
| Créer la commande d'impression du menu | À faire | Commander l'impression d'un menu | P1 | Développeur back-end | 5 | L'utilisateur peut commander l'impression d'un menu. La commande est enregistrée avec les informations nécessaires au traitement par Qwenta. | Express.js, PostgreSQL, endpoint de commande d'impression, association avec le PDF généré et le menu concerné. |
| Afficher les menus précédents | À faire | Afficher les menus précédents | P1 | Développeur front-end | 3 | L'utilisateur peut consulter la liste de ses menus précédemment créés depuis son espace connecté. | React, appel API, affichage en liste ou cartes, récupération des menus associés à l'utilisateur connecté. |
| Permettre la modification d'un menu existant | À faire | Accéder aux menus existants | P1 | Développeur front-end | 5 | L'utilisateur peut rouvrir un menu existant, modifier son contenu et sauvegarder les changements. | React, React Router, chargement des données existantes, formulaire prérempli, appels API PUT. |
| Permettre la suppression d'un menu | À faire | Supprimer un menu précédents | P1 | Développeur back-end | 3 | L'utilisateur peut supprimer un menu dont il est propriétaire, après confirmation. | Express.js, API REST, suppression sécurisée, vérification des droits utilisateur, PostgreSQL. |
| Gérer les informations utilisateur | À faire | Gérer ses Infos utilisateur | P2 | Développeur front-end | 3 | L'utilisateur peut consulter ses informations de compte et les informations liées à son restaurant. | React, page profil utilisateur, récupération des données via API, affichage sécurisé des informations. |
| Gérer la déconnexion | À faire | Se déconnecter | P2 | Développeur front-end | 2 | L'utilisateur peut se déconnecter depuis l'espace privé et revient à une page publique après déconnexion. | Auth0, React, bouton de déconnexion, redirection après logout, suppression de l'état utilisateur côté interface. |
| Préparer l'export vers Deliveroo | À faire | Exporter sur Deliveroo | P2 | Développeur back-end | 8 | Le système prépare les données du menu dans un format compatible avec une intégration Deliveroo, sous réserve de l'accès à l'API partenaire. | Express.js, transformation des données Menu Maker, Deliveroo Menu API, gestion des erreurs et sécurisation des clés API. |
| Préparer le partage Instagram | À faire | Partager sur Instagram | P2 | Développeur back-end | 8 | Le système permet de préparer un visuel du menu compatible avec un partage sur Instagram. | Express.js, génération d'image JPEG/PNG à partir du menu, Instagram Graph API, gestion des autorisations et des limites API. |
| Ajouter le branding restaurateur | À faire | Identité visuelle du restaurateur | P3 | Développeur front-end | 5 | L'utilisateur peut ajouter des éléments d'identité visuelle de son restaurant afin de personnaliser davantage ses menus. | React, upload de logo ou visuel, gestion des couleurs, stockage des informations de branding, affichage dans la prévisualisation du menu. |

💡 Notice de lecture
Ce Kanban présente la planification technique du développement de Menu Maker by Qwenta. Chaque carte correspond à une tâche technique issue des user stories du projet.
Les priorités P1, P2 et P3 reprennent le niveau de priorité des user stories. L'assignation précise si la tâche relève principalement du front-end ou du back-end.
Les story points indiquent la complexité relative d'une tâche : effort nécessaire, difficulté technique, risques et incertitudes.
Échelle utilisée inspirée de la suite de Fibonacci :
1 = très simple
2 = simple
3 = moyen
5 = complexe
8 = très complexe